In our work, the following rt-patch and kernel version were used:

```
$ rt patch-version: patch-5.6.19-rt12.patch

$ Linux Kernel version: linux-5.6.19
```

Test suite: rt-tests

The rt-tests test suite contains programs to test various real-time Linux features; more details are available at *https://wiki.linuxfoundation.org/realtime/documentation/howto/tools/rt-tests*. The step-by-step procedure to install the rt-tests suite from the source is given below.

First, you need to install the libraries:

```
$ sudo apt-get install build-essential libnuma-dev
```

Next, clone the code and build from the source:

```
$ git clone git://git.kernel.org/pub/scm/utils/rt-tests/rt-tests.git
$ cd rt-tests
$ git checkout stable/v1.0
$ make all
$ make install
```

To avoid large booting times, it's very important to call the *make* command with the following option:

```
$ make INSTALL_MOD_STRIP=1
```

This helps to reduce the size of the suite from 170GB to 20GB approximately. The reason is that the *make* option has removed the debug statements from the modules.

Setting up the test environment

To set up a test environment where there is competition for system resources, some sort of test load needs to be run to be a determinism disturbance generator or DDG. A commonly used DDG is to start a kernel build using the parallel *make* feature to run many simultaneous compiles.

Setting up kernel build load: The following steps will start the kernel build; you can open another terminal and run the tests there.

```
$ wget http://www.kernel.org/pub/linux/kernel/vx.y/linux-x.y.z.tar.bz2
$ tar xf linux-x.y.z.tar.bz2
$ cd linux-x.y.z.tar.bz2
$ mkdir /tmp/build
$ make O=/tmp/build mrproper
$ make O=/tmp/kbuild allmodconfig
$ make O=/tmp/build -j4
```

Setting up signaltest: The signaltest utility was written by Thomas Gleixner to measure the latencies involved in a simple message passing mechanism, the *pthread* signals.

By default, two threads will be created. These threads will continuously wait for a signal from the other thread, immediately sending a signal back and restarting the loop. At each loop, a measurement of the signal delivery latency is done, and statistics about the measurements are shown on the terminal.

The command used to start signaltest is:

```
$ ./signaltest -p 99
```

The 'p' option in the above command execution assigns the number that follows (in this case, 99) as the priority of the highest thread.

Setting up sigwaittest: The sigwaittest utility was written by Carsten Emde to measure the latency between sending and receiving a signal between two threads or two processes via a fork. The program sigwaittest creates a pair of threads or two processes (via a fork), synchronises them via signals, and calculates the latency between sending a signal and returning from *sigwait()*.

The command used to start sigwaittest is:

```
$ ./sigwaittest -a -t -p99 -i100
```

- The 'a' option in the above command specifies on which processor the utility should be run. If nothing is specified, it's run on the current processor.
- The 't' option is used to set the number of test threads (default is 1, if this option is not given). If the 't' option is used and number of threads is not specified, the number of threads is taken as the number of available CPUs.
- The 'p' option is used to set the priority of the process. So, in the above command, the priority of the process is 99.
- The 'i' option is used to set the base interval of a thread in microseconds. The interval specifies the time the receiver thread waits for the signal from the sender thread.

Setting up ptsematest: The ptsematest utility was written by Carsten Emde to measure the latency of interprocess communication with POSIX mutex. The program creates a pair of threads that are synchronised using *pthread_mutex_unlock ()* / *pthread_mutex_lock ()*, and the time difference between getting and releasing the lock is measured as the latency.

The command used to start ptsematest is:

```
$ ./ptsematest -a -t -p99 -i100 -d0
```

The explanations for the options a, t, p and i are given while describing sigwaittest. The same explanations are applicable for all the tests listed below.

The 'd' option is used to set the distance of thread intervals in microseconds. As an example, if we are to create a couple of thread pairs and the distance is set to say 500µs, then the first thread pair interval is 100, the second thread pair interval becomes 100 + 500, the third interval becomes 100 + 500 + 500, and so on. In the above command, the distance is zero, which means all the thread pairs will have the same interval.